

Overview TCP Labview to Pi Communication Procedure

- Raspberry Pi acts as a TCP Server
- LabView acts as TCP client
- Button Press command is sent via string
- Pi receives command and sends a response back

List of Command strings sent to Pi

- UP, SELECT, DOWN, LEFT, RIGHT, BACK, MENU, RUN_OCR

To define string ends add new lines:

- UP/n, SELECT/n, etc.
- Then to decode new lines: On PI
 - `command = data.decode().strip()`

To Send OCR result ready from Raspberry Pi to Labview:

A continual listening is conducted to receive commands from labview. In order to also be able to send commands independently, the Pi needs to timeout its listening periodically and check if an OCR ready command needs to be sent.

Create a TCP Server on Raspberry Pi: (File saved as tcp_server.py)

```
import socket
```

```
HOST = '0.0.0.0' # Listen on all interfaces  
PORT = 5000
```

```
#Listening and waiting for tcp connection from client  
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)  
server.bind((HOST, PORT))  
server.listen(1)
```

```
print("Waiting for connection...")
```

```
#connection from client found  
conn, addr = server.accept()  
print(f"Connected by {addr}")
```

```
#Timeout needed to ensure Pi has time to send a OCR ready response independent
#from a Labview Command
conn.settimeout(0.1)
```

```
while True:
```

```
#checking if ocr_result is ready from camera vision system
if ocr_result:
    response = "OCR ready\n"
    conn.sendall(response.encode())
```

```
#if there is a Labview command, capture it with data
try:
    data = conn.recv(1024) #1024 byte string limit
```

```
    If data:
```

```
#decodes \n to end string
    command = data.decode().strip()
```

```
    print("Received:", command)
```

```
# Example command handling
    if command == "UP":
        response = "UP Button Pressed\n"
```

```
    elif command == "SELECT":
        response = "SELECT Button Pressed\n"
```

```
    elif command == "DOWN":
        response = "DOWN Button Pressed\n"
```

```
    elif command == "LEFT":
        response = "LEFT Button Pressed\n"
```

```
    elif command == "RIGHT":
        response = "RIGHT Button Pressed\n"
```

```
    elif command == "BACK":
        response = "BACK Button Pressed\n"
```

```
elif command == "MENU":
    response = "MENU Button Pressed\n"

elif command == "RUN_OCR":
    response = "Running OCR\n"

elif command == SEND_OCR_RESULT:
    response = "Sending OCR\n"

else:
    response = "Unknown Command\n"

conn.sendall(response.encode())

#ensures that if no data is found in data = conn.recv(1024) line then the program
#doesn't timeout
except socket.timeout:
    pass

conn.close()
```

To Run the TCP Server on Raspberry Pi:

Call this line of code on the main terminal of the Raspberry Pi. This will continuously run the server set up previously (tcp_server.py).

- `python3 tcp_server.py`

Continued TCP Connection:

We will most likely write several button press commands at once to navigate to a certain screen. This series of commands will be sent in just 1 TCP Connection. Steps 2 and 3 will be repeated as necessary in a loop.

Bytes to Read Command:

In the Labview TCP Client portion of the example, we need to wait for a response from the Pi before continuing to the next command. Therefore, every time a command needs to be read, after TCP Read, is a Bytes to Read. This ensures that the program will not continue until the exact number of bytes specified is read. For example, if BACK Button Pressed\n needs to be read from the Pi, then the program will not continue until 20 bytes are received from the Pi.

LABVIEW TCP Client Example:

#Set Pi IP Address and Port

TCP Open Connection

IP: 192.168.1.50

Port: 5000

#Heat_Pump(Only) Screen Navigation

#1. Back

TCP Write

Data = "BACK\n"

#Bytes to Read should = BACK Button Pressed\n

TCP Read

Bytes to Read = 20

#2. Menu

TCP Write

Data = "MENU\n"

#Bytes to Read should = MENU Button Pressed\n

TCP Read

Bytes to Read = 20

#3. Select

TCP Write

Data = "SELECT\n"

#Bytes to Read should = SELECT Button Pressed\n

TCP Read

Bytes to Read = 22

#4. Left

TCP Write
Data = "LEFT\n"
#Bytes to Read should = LEFT Button Pressed\n
TCP Read
Bytes to Read = 20

#5. Left
TCP Write
Data = "LEFT\n"
#Bytes to Read should = LEFT Button Pressed\n
TCP Read
Bytes to Read = 20

#6. Select
TCP Write
Data = "SELECT\n"
#Bytes to Read should = SELECT Button Pressed\n
TCP Read
Bytes to Read = 22

#7. Down
TCP Write
Data = "DOWN\n"
#Bytes to Read should = DOWN Button Pressed\n
TCP Read
Bytes to Read = 20

#8. Down
TCP Write
Data = "DOWN\n"
#Bytes to Read should = DOWN Button Pressed\n
TCP Read
Bytes to Read = 20

#9. Select
TCP Write
Data = "SELECT\n"
#Bytes to Read should = SELECT Button Pressed\n
TCP Read
Bytes to Read = 22

#10. Right
TCP Write
Data = "RIGHT\n"
#Bytes to Read should = RIGHT Button Pressed\n
TCP Read
Bytes to Read = 21

#11. Right
TCP Write
Data = "RIGHT\n"
#Bytes to Read should = RIGHT Button Pressed\n
TCP Read
Bytes to Read = 21

#12. Prompt Pi to run OCR
TCP Write
Data = "RUN_OCR\n"
#Bytes to Read should = Running OCR\n
TCP Read
Bytes to Read = 12

#13. Wait for Pi to send OCR ready response
#Bytes to Read should = OCR ready\n
TCP Read
Bytes to Read = 10

#14. Prompt Pi to send OCR result
TCP Write
Data = "SEND_OCR_RESULT\n"
#Bytes to Read should = Sending OCR\n
TCP Read
Bytes to Read = 12

#Heat_Pump(Only) Screen Navigation Successful - Close TCP Connection
TCP Close Connection